Blast Radius

An agent that reviews Terraform pull requests — built to test whether agent machinery beats simply asking a model, and finding that it does not.

Result: nothing beat one prompt. A one-shot review of the diff scores **F1 0.784** at **$0.007** per pull request. Three of the six agentic additions built on top of it — the terraform plan, a multi-agent split, and tools with free exploration — made it **measurably worse**. The other three could not be distinguished from doing nothing.

A late variance check invalidated four of this project's own findings. That correction is the most useful thing here, and it is documented rather than buried.

- **report/changelog.md** — every stage, every prediction recorded before its run, and the four that were withdrawn
- **REPRODUCE.md** — clean-environment guide, exact commands, measured runtime and cost
- **report/video-script.md** — the 5-minute walkthrough

The problem

The user is whoever approves infrastructure pull requests on a small team — usually one or two people who own production and review Terraform written by everyone else.

The diff does not tell you what will happen. A one-line change can destroy a database. A variable default edited in one file can widen an IAM policy defined in another. Meanwhile the scanner in CI reports on the whole stack every time — **9.5 findings per pull request in this fixture, regardless of what changed** — so the team learned months ago to scroll past it. The tooling is simultaneously too noisy to read and too shallow to catch what causes incidents.

This is the last human checkpoint before production. Missing something costs downtime or a data-loss postmortem. The noise costs the checkpoint itself.

How this is measured

Scoring is set intersection on (resource address, category) across **15 labelled pull requests** carrying **10 real findings** and **7 findings a good reviewer suppresses**. No model grades another model anywhere in this project, so the numbers are deterministic given the same outputs.

The obvious version of this project scores itself against Checkov labels — and that version is dead on arrival, because then running Checkov scores 1.0 and the agent is an expensive wrapper. So the cases are grouped into four bands, two of which a scanner structurally cannot win:

Band	Cases	What it contains	The scanner
A	3	Real problems the scanner catches	catches — the agent must not regress
B	4	Correct-but-irrelevant findings	false positives — suppression is the skill
C	6	Cross-file, plan-transition and cost problems	structurally blind (scores 0.14)
D	2	Genuinely clean pull requests	does the agent invent problems?

A label with category `noise` is a finding a good reviewer suppresses. Reporting one costs precision. That is what stops any configuration from scoring well by reprinting the scanner.

Results

Configurations marked (n=4) were run four times. **The spread matters more than any single number** — this project learned that the hard way, and the baseline's own standard deviation is 0.078.

configuration	F1		$/PR
Raw Checkov — what CI runs today	0.052	9.5 findings per PR	$0
Checkov scoped to the change	0.320	the fair scanner baseline	$0
One prompt, Haiku 4.5 (n=4)	**0.784 ± 0.078**	**the best thing measured**	**$0.007**
One prompt, Sonnet 4.6	0.783	no better, 2× the price	$0.015
+ terraform plan (n=4)	0.674 ± 0.038	**worse** (–0.110, no overlap)	$0.009
+ multi-agent split (n=4)	0.653 ± 0.060	**worse** (–0.130, no overlap)	$0.026
+ tools, free exploration (n=4)	0.588 ± 0.042	**worst** (–0.196, no overlap)	$0.133
+ scanner output as claims	0.842	no measurable effect	$0.007
+ citation verification	0.900	no measurable effect, and free	$0.000
+ cost table	0.842	no measurable effect	$0.007
+ review memory	0.800	no measurable effect	$0.007

The interesting comparison is not against the scanner, which loses to everything. It is between the simplest model configuration and the elaborate ones. The elaborate ones lose.

Case 08 is the whole argument in one line of diff. A pull request enables encryption at rest on the production database — `storage_encrypted = false → true` — which is correct, is a genuine security fix, and destroys the database, because that attribute is immutable on RDS.

	on case 08
Checkov	flags the database — for missing log exports
One prompt	`data-loss` — *"will destroy the current database and create a new empty one"*
+ terraform plan	`reliability` — *"the application will lose connectivity… ten to thirty minutes"*

Shown a plan stating `delete` then `create`, the model reasons about the resource lifecycle and stops reasoning about what is inside it. The plan is strictly more accurate than the diff, and it made the review worse. The same recategorisation appeared independently in the tools configuration, which reached the plan by a different route.

What this project concluded

Context is not free, and it is not neutral. Adding it reframes the question. The plan made the review about resource lifecycles. A security scanner's output made it about security. Tools made it about the codebase rather than the change. Every context added to fix a blind spot also tells the model that blind spot is someone else's job now.

A single run is not a measurement, and this project nearly published four that were not. Six iterations were built, run once each, and compared against one baseline number of 0.900. Four regression stories were written, each with a plausible mechanism, each committed. Then the baseline was repeated three times and scored 0.737, 0.762 and 0.737 — the original was the best of four — and three of those stories evaporated.

Nothing about the method was careless in an obvious way: deterministic scoring, labels fixed before any run, no LLM judge, every prediction recorded in advance. The missing control was the cheapest available — running the same thing twice. **$0.33 and twelve minutes.** It should have been the second measurement in the project, not the twenty-second.

The trap is that a pipeline with structured outputs, fixed prompts and set-intersection scoring *looks* deterministic, which quietly suggests the model is too. On 15 cases carrying 10 findings, one finding moving is 0.05–0.10 of

F1. The noise floor was wider than five of the eight effects being measured.

Reproduction

Full guide in **REPRODUCE.md**. The harness half needs no AWS account and no money:

```
pip install -r requirements.txt
./plan.sh --all                            # ~4 min, every plan and diff
python tools/run_checkov.py --all --scoped  # ~6 min, the scanner baseline
python score.py --mode checkov-scoped       # expect F1 0.320
```

The short path to the main result, with Bedrock access — **4 minutes, $0.11**:

```
python tools/run_oneshot.py --all --model haiku              # the winner
python tools/run_oneshot.py --all --with-plan --model haiku  # the plan makes it worse
python score.py --mode oneshot-haiku    # ~0.78
python score.py --mode i1plan-haiku     # ~0.67
```

The project's central hypothesis was that the second would beat the first. Everything reproduces in about 3 hours for $10.44, which is what it cost.

Model access runs through Amazon Bedrock on the ordinary AWS credential chain. **There is no API key anywhere in this repository.**

Layout

```
infra/base/          fixture stack, 32 resources, no data sources
fixtures/            the converged state fixture
cases/case-NN-*/     overlay/ (the pull request), pr_description.md, labels.yaml
prompts/review.md    review instructions, shared by every stage
plan.sh              overlay -> plan.json + diff
score.py             findings x labels -> F1, band recall, per-band detail
tools/               harness: make_state, fixture_ids, planfilter, emit_plan,
                     make_oracle, pricing, model
                     runners: run_checkov, run_oneshot, run_agent_b2,
                     run_i5_memory, run_i6_multiagent, verify_citations
results/             plans, diffs, findings, trajectories (24 configurations)
report/              changelog, video script, per-mode metrics
```

A case's `overlay/` holds whole `.tf` files replacing their base version; `plan.sh` copies base, applies the overlay, and diffs the two to produce the pull request a reviewer would see.

The stages share one runner and one prompt file. I1, I2 and I4 are flags on `run_oneshot.py`, so the model, instructions, output schema and code path are identical between them and the baseline, and the added context is genuinely the only variable. Copying the prompt into separate scripts would have made wording drift indistinguishable from the effect being measured.

Method notes

No AWS account is needed for the harness. The provider uses dummy credentials with every API-reaching call disabled, so `terraform plan` runs offline. The constraint: no data sources anywhere in the fixture, since those do hit the API. Anything normally looked up is a variable.

Prior state is synthesized, not applied. `terraform plan` against empty state reports everything as `create`, so a *replace* or *destroy* cannot occur — and those are exactly the transitions a scanner cannot see. `tools/make_state.py` derives state from the plan's own `planned_values`, fills the identifiers only known after apply, then converges: write, re-plan, fold back, repeat until the plan is a clean no-op. Four rounds. Committed

as `fixtures/base.tfstate`; you never need to run it.

One consequence, documented because it is a real limitation: `aws_db_instance` always reports an in-place update against synthesized state even when every attribute before and after is identical. `tools/planfilter.py` drops updates whose `before` equals `after` — narrow, provable, and unable to hide a real finding, including case 08's replace, which does change an attribute.

Scoring the scanner fairly. Checkov classifies by its own check ids, so the mapping to this project's categories lives in `tools/run_checkov.py` and is published rather than buried, because it decides how well the baseline does. Nothing maps to `data-loss`, `guardrail`, `cost` or `reliability` — Checkov has no check for any of them. That absence is the finding, not an oversight.

Three scoring rules exist because the first version of each was wrong. All three corrections raised a baseline rather than the agent:

- Findings match on address **and** category. An earlier address-only rule credited the scanner for naming the right resource for unrelated reasons, inflating its band C recall from 0.14 to 0.43.
- An input variable has three legitimate names, and only one was accepted, so case 12 scored zero for two configurations that had both found the right problem. `norm()` now strips `var.` and `variable.` prefixes.
- `noise` labels match on **address alone** — raising that resource at all is the error, whatever category it is filed under. Matching noise on category too let a miscategorised parrot through, and reported 6 of 6 noise suppressed when the true figure was 1 of 6.

Disclosure

The fixture stack, the cases and the labels are all written for this project. Labels were committed before any model run and each cites the file line or plan action justifying it; every correction made to them afterwards is recorded in the changelog.

Two deliberate deviations from the original plan, both in the changelog: the "general agent with shell access" baseline was built with a confined tool surface because these runs happen on a workstation with live cloud credentials, and no model-authored command is executed anywhere in this project.

Nothing here corresponds to real infrastructure. The AWS account id in the fixtures is all zeros and the provider credentials are the literal string `test`.